```
DROP DATABASE IF EXISTS Student_enrollment;
CREATE DATABASE Student_enrollment;
USE Student_enrollment;

-- Create table STUDENT
CREATE TABLE STUDENT (
   regno VARCHAR(40) PRIMARY KEY,
   name VARCHAR(100),
   major VARCHAR(100),
   bdate DATE
);

-- Create table COURSE
CREATE TABLE COURSE (
   course INT PRIMARY KEY,
   cname VARCHAR(100),
   dept VARCHAR(100)
);

-- Create table ENROLL
CREATE TABLE ENROLL (
   regno VARCHAR(40),
   course INT,
   sem INT,
   marks INT,
   FOREIGN KEY (regno)
      REFERENCES STUDENT (regno)
      ON DELETE CASCADE,
   FOREIGN KEY (course)
      REFERENCES COURSE (course)
      ON DELETE CASCADE
);

-- CREATE table TEXT
```

```
CREATE TABLE TEXT (
   book_ISBN INT PRIMARY KEY,
   title VARCHAR(100),
   publisher VARCHAR(100),
   author VARCHAR(100)
);

-- Create table BOOK_ADOPTION
CREATE TABLE BOOK_ADOPTION (
   course INT,
   sem INT,
   book_ISBN INT,
   FOREIGN KEY (course)
      REFERENCES COURSE (course)
      ON DELETE CASCADE,
   FOREIGN KEY (book_ISBN)
      REFERENCES TEXT (book_ISBN)
      ON DELETE CASCADE
);

INSERT INTO STUDENT VALUES
("01HF235", "Student_1", "CSE", "2001-05-15"),
("01HF354", "Student_2", "Literature", "2002-06-10"),
("01HF254", "Student_3", "Philosophy", "2000-04-04"),
("01HF653", "Student_4", "History", "2003-10-12"),
("01HF234", "Student_5", "Computer Economics", "2001-10-10"),
("01HF699", "Student_6", "Computer Economics", "2001-10-10");

INSERT INTO COURSE VALUES
(001, "DBMS", "CS"),
(002, "Literature", "English"),
(003, "Philosophy", "Philosphy"),
(004, "History", "Social Science"),
(005, "Computer Economics", "CS");
```

```
INSERT INTO ENROLL VALUES
("01HF235", 001, 5, 85),
("01HF354", 002, 6, 87),
("01HF254", 003, 3, 95),
("01HF653", 004, 3, 80),
("01HF234", 005, 5, 75),
("01HF699", 001, 6, 90);

INSERT INTO TEXT VALUES
(241563, "Operating Systems", "Pearson", "Silberschatz"),
(532678, "Complete Works of Shakesphere", "Oxford", "Shakesphere"),
(453723, "Immanuel Kant", "Delphi Classics", "Immanuel Kant"),
(278345, "History of the world", "The Times", "Richard Overy"),
(426784, "Behavioural Economics", "Pearson", "David Orrel"),
(469691, "Code with Fun", "Tim David", "David Warner"),
(767676, "Fun & philosophy","Delphi Classics", "Immanuel Kant");

INSERT INTO BOOK_ADOPTION VALUES
(001, 5, 241563),
(002, 6, 532678),
(003, 3, 453723),
(004, 3, 278345),
(001, 6, 426784),
(001, 5, 469691),
(003, 6, 767676);

-- 1. Demonstrate how you add a new text book to the database and make this book be
-- adopted by some department.

-- Insert into TEXT table
INSERT INTO TEXT VALUES
(987654, "New Textbook", "New Publisher", "New Author");
-- check
```

```
SELECT * FROM TEXT;


INSERT INTO BOOK_ADOPTION VALUES (001, 5, 987654);
SELECT * FROM BOOK_ADOPTION;

-- 2. Produce a list of text books (include Course #, Book-ISBN, Book-title) in the alphabetical
-- order for courses offered by the ‘CS’ department that use more than two books.


SELECT
   BA.course, BA.book_ISBN, T.title
FROM
   BOOK_ADOPTION BA
   JOIN COURSE C ON BA.course = C.course
   JOIN TEXT T ON BA.book_ISBN = T.book_ISBN
WHERE
   C.dept = 'CS'
   AND BA.course IN (
      SELECT course
      FROM BOOK_ADOPTION
      GROUP BY course
      HAVING COUNT(book_ISBN) > 2
   )
ORDER BY T.title;



-- 3. List any department that has all its adopted books published by a specific publisher.
SELECT DISTINCT dept
FROM COURSE
WHERE dept IN(
            SELECT dept FROM COURSE JOIN BOOK_ADOPTION
   USING(course) JOIN TEXT USING(book_ISBN)
   WHERE publisher='Delphi Classics'
)
AND
```

```
dept NOT IN(
        SELECT dept FROM COURSE JOIN BOOK_ADOPTION
  USING(course) JOIN TEXT USING(book_ISBN)
  WHERE publisher != 'Delphi Classics'
);

-- 4. List the students who have scored maximum marks in ‘DBMS’ course.
SELECT S.regno, S.name, E.marks
FROM STUDENT S
JOIN ENROLL E ON S.regno = E.regno
JOIN COURSE C ON E.course = C.course
WHERE C.cname = 'DBMS'
ORDER BY E.marks DESC
LIMIT 1;

-- 5. Create a view to display all the courses opted by a student along with marks obtained.
CREATE OR REPLACE VIEW StudentCourses AS
SELECT regno, course, cname, marks
FROM ENROLL
JOIN COURSE USING(course);

-- check
SELECT * FROM StudentCourses;

-- 6. Create a trigger that prevents a student from enrolling in a course if the marks
-- prerequisite is less than 40.

DELIMITER //
CREATE TRIGGER PreventEnrollment
BEFORE INSERT ON ENROLL
FOR EACH ROW
BEGIN
  IF NEW.marks < 40 THEN
    SIGNAL SQLSTATE '45000'
```

```
      SET MESSAGE_TEXT = 'Marks prerequisite not met for enrollment';
   END IF;
END;
//
DELIMITER ;

-- check trigger
INSERT INTO STUDENT (regno, name, major, bdate)
VALUES ('01HF999', 'John Doe', 'Computer Science', '2000-01-01');

INSERT INTO ENROLL (regno, course, sem, marks)
VALUES ('01HF999', 1, 7, 32);
```